LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 05 – RELASI ANTAR KELAS

BILQIS KEYRA AZAHRA

103132400043

PS01DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dalam Pemrograman Berorientasi Objek (OOP), kelas-kelas saling berinteraksi membentuk relasi yang dimodelkan dalam Class Diagram. Relasi ini terbagi menjadi tiga jenis utama. Asosiasi adalah hubungan komunikasi dasar antar kelas, di mana suatu objek dapat memanggil method objek lain tanpa menjadikannya sebagai atribut. Agregasi merupakan hubungan "bagian dari" (part-of) yang bersifat longgar, di mana objek bagian tetap bisa berdiri sendiri meskipun objek utamanya dihapus. Sementara itu, Komposisi adalah bentuk agregasi yang lebih kuat dan ketat; objek bagian tidak akan ada tanpa objek utama, sehingga jika objek utama dihancurkan, objek bagian akan ikut musnah.

B. Guided

1. Agregasi

Jurusan.java

```
package com.mycompany.guided.agregasi;

/**
 *
 * @author ACER
 */
public class Jurusan {
     private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    // constractor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }
    // behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }
    public void displayMahasiswa(){
        System.out.println("Kode jurusan: " +  this.kodeJurusan);
        System.out.println("Nama jurusan: " + this.namaJurusan);

        System.out.println("Daftar mahasiswa: ");

        for(int i = 0; i<jumlahMahasiswa; i++){
            System.out.println(i+1 + ". " + daftarMahasiswa[i].getNIM() + " " +
daftarMahasiswa[i].getNama());
        }

    }
    // getter
```

```
    public String getkodeJurusan(){
        return kodeJurusan;
    }
    public String getnamaJurusan(){
        return namaJurusan;
    }

    // setter
    public void setkodeJurusan(String kodejurusan){
        this.kodeJurusan = kodejurusan;
    }
    public void setnamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
```

Mahasiswa.java

```
package com.mycompany.guided.agregasi;

/**
 *
 * @author ACER
 */
public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    // getter
    public String getNIM(){
        return NIM;
    }
    public String getNama(){
        return Nama;
    }

    // setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }
    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

2. Asosiasi

Dosen.java

```
package com.mycompany.guided.asosiasi;

/**
 *
 * @author ACER
 */
public class Dosen {

    private String kodeDosen;
    private String namaDosen;
    //Setter

    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    //Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");
        Dosen d = new Dosen();
        d.giveScore(m, 90);
        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Mahasiswa.java

```
package com.mycompany.guided.asosiasi;

/**
 *
 * @author ACER
 */
public class Mahasiswa {

    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
    }

    public String getNama() {
        return this.nama;
    }

    public String getNim() {
        return this.nim;
    }

    public int getNilai() {
        return this.nilai;
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ mavenproject1 ---
Kode jurusan: SD
Nama jurusan: Sains Data
Daftar mahasiswa:
1. 103112400148 Rendhi
2. 103112400149 John
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.863 s
Finished at: 2026-10-19T21:21:52+07:00
```

Activate Winc

3. Komposisi

Komputer.java

```
package com.mycompany.guided.komposisi;

/**
 *
 * @author ACER
 */
public class Komputer {
    private Processor processor;
   private Monitor monitor;
   private Mouse mouse;

   public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor, String
brandMouse, String tipeMouse){
       this.processor = new Processor(brandCpu, kecepatanCpu);
       this.monitor = new Monitor(brandMonitor);
       this.mouse = new Mouse(brandMouse, tipeMouse);
   }

   public void displaySpecKomputer() {
       System.out.println("=== SPESIFIKASI KOMPUTER ===");
       this.processor.displaySpec();
       this.monitor.DisplaySpec();
       this.mouse.displaySpec();
   }
}
```

Monitor.java

```
package com.mycompany.guided.komposisi;

/**
 *
 * @author ACER
 */
public class Monitor {
    private String brand;

   public Monitor(String brand){
       this.brand = brand;
   }

   public void DisplaySpec() {
       System.out.println(this.brand);
   }
}
```

Mouse.java

```
public class Mouse {
    private String brand, tipe;
```

```
    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

Proccessor.java

```
public class Processor {
     private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Output :

```
Compiling 14 source files with javac [debug release 26] to target\classes

--- exec:3.6.3:exec (default-cli) @ mavenprojectl ---
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  7.130 s
Finished at: 2026-10-19T21:24:42+07:00
------------------------------------------------------------------------
```

Activate Windo

C. Unguided

1. Agregasi

Jawaban :

Pesanan.java

```
package unguided.agregasi;

import unguided.komposisi.ItemPesanan;
import unguided.komposisi.RincianHarga;
```

```
/**
 *
 * @author ACER
 */
public class Pesanan {
    private int nomorMeja;
   private ItemPesanan[] daftarItem;
   private int jumlahItem;

   public Pesanan(int nomorMeja, int kapasitas) {
      this.nomorMeja = nomorMeja;
      this.daftarItem = new ItemPesanan[kapasitas];
      this.jumlahItem = 0;
   }

   public void tambahItem(ItemPesanan item) {
      if (this.jumlahItem < this.daftarItem.length) {
         this.daftarItem[this.jumlahItem] = item;
         this.jumlahItem++;
      } else {
         System.out.println("Kapasitas pesanan penuh!");
      }
   }

   public void tampilkanPesanan() {
      System.out.println("Nomor Meja: " + this.nomorMeja);
      for (int i = 0; i < this.jumlahItem; i++) {
         ItemPesanan item = this.daftarItem[i];
         RincianHarga r = item.getRincian();
         System.out.println((i + 1) + ". " + item.getNamaItem()
               + " - Rp " + r.getHargaTotal()
               + " (Harga: " + r.getHargaDasar()
               + ", Pajak: " + r.getPajak() + ")");
      }
   }

   public int getNomorMeja() {
      return this.nomorMeja;
   }

   public ItemPesanan[] getDaftarItem() {
      return this.daftarItem;
   }

   public int getJumlahItem() {
      return this.jumlahItem;
   }
}
```

2. Asosiasi

Jawaban :

Kasir.java

```
package unguided.asosiasi;

import unguided.agregasi.Pesanan;
import unguided.komposisi.ItemPesanan;

/**
 *
 * @author ACER
 */
public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] items = pesanan.getDaftarItem();
        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            totalSubtotal += items[i].getRincian().getHargaDasar();
            totalPajak += items[i].getRincian().getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

3. Komposisi

Jawaban :

ItemPesanan.java

```
package unguided.komposisi;

/**
 *
 * @author ACER
```

```
 */
public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian; // Komposisi: objek internal

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        // Komposisi: RincianHarga di-instansiasi di dalam constructor
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

RincianHarga.java

```
package unguided.komposisi;

/**
 *
 * @author ACER
 */
public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10; // pajak 10%
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ mavenproject1 ---
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
------------------------------------------------------------------------
```

Penjelasan:

Berdasarkan output program, sistem berhasil mengimplementasikan relasi antar kelas dengan baik. Bagian pertama menampilkan daftar pesanan untuk Meja 4 beserta rincian harga dan pajak per item, yang membuktikan bahwa interaksi antar objek dalam menghitung data berjalan lancar. Selanjutnya, pada bagian struktur pembayaran, objek Kasir berhasil membaca data dari objek Pesanan untuk menghitung total keseluruhan. Program menampilkan jumlah item, subtotal sebesar Rp 65.000, total pajak Rp 6.500, dan total bayar akhir sebesar Rp 71.500. Secara keseluruhan, output ini membuktikan bahwa enkapsulasi serta hubungan ketergantungan antar kelas (Kasir, Pesanan, ItemPesanan, dan RincianHarga) telah dirancang dan dieksekusi dengan sukses sesuai fungsinya.

D. Kesimpulan

Berdasarkan pelaksanaan Praktikum Modul 5 mengenai Relasi Antar Kelas, dapat disimpulkan bahwa perancangan perangkat lunak berorientasi objek. Melalui implementasi kode, praktikan dapat membedakan tiga jenis relasi utama. Hubungan Asosiasi diterapkan ketika dua kelas hanya perlu saling berkomunikasi melalui pemanggilan method tanpa adanya keterikatan kepemilikan atribut, seperti pada relasi antara kelas Kasir dan Pesanan. Selanjutnya, hubungan Agregasi digunakan ketika sebuah kelas bertindak sebagai wadah bagi kelas lainnya, namun objek yang menjadi bagian tersebut tetap memiliki siklus hidup yang mandiri. Hal ini diimplementasikan pada kelas Jurusan yang menampung objek Mahasiswa, di mana keberadaan objek Mahasiswa tidak sepenuhnya bergantung pada objek Jurusan. Terakhir, penerapan access modifier private pada atribut kelas terbukti sangat penting untuk menjaga enkapsulasi data. Dengan demikian, penguasaan terhadap konsep asosiasi, agregasi, dan komposisi memungkinkan praktikan untuk memetakan kebutuhan sistem nyata ke dalam arsitektur kode Java yang logis, terstruktur, serta mudah untuk dikembangkan di masa depan.